
Length Generalization Bounds for Transformers

Andy Yang¹ Pascal Bergsträßer² Georg Zetsche³ David Chiang¹ Anthony W. Lin^{2,3}

Abstract

Length generalization is a key property of a learning algorithm that enables it to make correct predictions on inputs of any length, given finite training data. To provide such a guarantee, one needs to be able to compute a length generalization bound, beyond which the model is guaranteed to generalize. This paper concerns the open problem of the computability of such generalization bounds for C-RASP, a class of languages which is closely linked to transformers. A positive partial result was recently shown by [Chen et al.](#) for C-RASP with only one layer and, under some restrictions, also with two layers. We provide complete answers to the above open problem. Our main result is the non-existence of computable length generalization bounds for C-RASP (already with two layers) and hence for transformers. To complement this, we provide a computable bound for the positive fragment of C-RASP, which we show equivalent to fixed-precision transformers. For both positive C-RASP and fixed-precision transformers, we show that the length complexity is exponential, and prove optimality of the bounds.

1. Introduction

The past few years have witnessed intensive research efforts to provide theoretically sound analyses of transformers (e.g. [Strobl et al., 2024](#)). While initial efforts focused primarily on transformer expressivity, more recent efforts ([Varre et al., 2025](#); [Wang et al., 2025](#); [Chen et al., 2025](#); [Huang et al., 2025](#)) have delineated what is efficiently learnable by transformers, particularly in terms of *length generalization*. Intuitively, length generalization is when a model can be

trained to accurately process strings of *any length*, given only a finite training sample of strings with *bounded length*. Understanding when transformers can length-generalize has important practical implications. On the one hand, computational and data constraints limit the sequence lengths that are seen during training; on the other hand, processing large contexts and reasoning over long chains-of-thought create a need for strong capabilities even on long inputs.

Empirical studies have found that length generalization in transformers can vary from problem to problem ([Zhou et al., 2024a](#)) through mechanisms that remain unclear. Theoretical work has begun to provide some characterizations of the formal languages which transformers length-generalize on. These characterizations are largely based on the expressivity of RASP, a programming language which was designed to capture the expressive power of transformers ([Weiss et al., 2021](#)). [Zhou et al. \(2024a\)](#) conjectured that transformers would length-generalize on a language if and only if it had a short solution in the RASP variant RASP-L, and provided empirical evidence to support this claim. Later, [Huang et al. \(2025\)](#) formalized this conjecture by proving that transformers are guaranteed to length-generalize under an idealized inference procedure for exactly those languages which are definable in C-RASP, a RASP variant expressively equivalent to transformers with fixed precision outside of attention ([Yang et al., 2025](#)). However, as pointed out by [Chen et al. \(2025\)](#), the results of [Huang et al.](#) only guarantee learnability *in the limit* ([Gold, 1967](#)), that is, without providing any quantitative bounds on the training resources needed to learn a given language.

For an example of how quantitative bounds lead to real-world impacts, we can look at scaling laws for language models. Language models are known to reliably follow scaling laws, which quantitatively describe how the test loss of a model relates to the size of the model and the training data ([Kaplan et al., 2020](#)). These laws can be used to derive the optimal model size and dataset size to minimize loss, given a fixed compute budget ([Hoffmann et al., 2022](#)). While these scaling laws may effectively predict loss, they fail to predict length generalization. A growing body of evidence shows that length generalization is often wholly independent of conventional scaling laws; in many cases, increasing the model size or number of training examples are not the critical factors which determine length generalization ([Anil](#)

¹University of Notre Dame, USA ²University of Kaiserslautern-Landau, Germany ³Max-Planck Institute for Software Systems, Germany. Correspondence to: Andy Yang <ayang4@nd.edu>, Pascal Bergsträßer <bergstraesser@cs.uni-kl.de>, David Chiang <dchiang@nd.edu>, Anthony Lin <awlin@mpi-sws.org>.

et al., 2022; Nye et al., 2022). As an example, Nogueira et al. (2021) find that models ranging from 50M to 3B parameters could not add 20-digit numbers when trained on addition of 15-digit numbers. At the same time, length generalization could be observed after training on 30 digits and testing on 60 digits, but increasing the amount of data past a critical threshold did not make much of a difference. A distinct theoretical framework is thus needed to provide quantitative guarantees for length generalization in transformers.

To provide a framework for these formal guarantees, Chen et al. (2025) formulated the notion of *non-asymptotic length generalization*, which asks for a *computable* bound N such that a learning algorithm only requires labeled training data consisting of strings of length up to N in order to correctly classify any test data containing strings of length greater than N . Such a learner is guaranteed to terminate, since N is computable and there are finitely many strings of length up to N . This notion of non-asymptotic length generalization is connected to the classical framework of *exact learning* (Angluin, 1988) with only membership queries, as well as learning minimal hypotheses, which was already studied by Gold for finite automata (Gold, 1978). In fact, Chen et al. show that non-asymptotic length generalization is equivalent to finite identification (Gold, 1967, p. 457) when the latter is allowed to take into account the complexity of the ground-truth hypothesis.

We adopt this general learning framework in order to provide length generalization guarantees while abstracting away from the training dynamics of specific gradient-based algorithms, which are notoriously difficult to analyze. This framework assumes the best-case scenario of having access to all possible examples up to a specified length and optimizing perfectly on those examples. Thus, if length generalization is proven to be difficult under these favorable conditions, then it should be even harder for SGD on limited data (as in practice). In short, we have proven a bound that would still apply even if learning dynamics were taken into account.

To study non-asymptotic length generalization in transformers, Chen et al. (2025) proved computable length generalization bounds for C-RASP with one layer and C-RASP with two layers and additional restrictions (see Appendix A.1). These bounds are (respectively) polynomial and exponential in the maximum absolute value of constants appearing in the C-RASP program. For example, using this, one can surmise that learning the MAJORITY language $\{w \in \{a, b\}^* : w \text{ has more } a\text{'s than } b\text{'s}\}$ using C-RASP programs of depth 1 requires only strings up to length quadratic in the absolute value of constants. Similarly, Izzo et al. (2026) established bounds on the length at which length generalization occurs in one- and two-layer transformers with some additional modifications (see Appendix A.2). These results

left open whether general C-RASP programs and transformers with more than two layers also admit non-asymptotic length generalization.

Contributions. The main contribution of this paper is to answer the open problem of the non-asymptotic length generalization of transformers and C-RASP in the negative.

Theorem 1.1 (Informal version of Corollary 3.6). *There is no terminating algorithm for perfectly learning a C-RASP program P (given an upper bound on the size of P), even if P only has depth two. Thus, no such algorithm exists for transformers of depth two or beyond.*

The consequence for transformers is a corollary of the depth-preserving equivalence between C-RASP and transformers (Yang et al., 2025). In particular, the lengths of strings necessary for length generalization must grow faster than any computable function.

To complement this uncomputability result, our secondary contribution is to provide a tight computable (in fact, *exponential*) length generalization bound for the positive fragment C-RASP₊ of C-RASP, which is expressively equivalent to fixed-precision transformers. This is a natural restriction (Li & Cotterell, 2025) since real-world transformers are implemented on a device that supports only a fixed finite precision (i.e. floating-point computation).

Theorem 1.2 (Informal version of Theorem 4.7). *To perfectly learn a C-RASP₊ program P (given an upper bound on the size of P), it is sufficient to train on strings with length exponential in the size of P , and, in the worst case, it is necessary to see at least one string of exponential length. The same bound holds for a transformer T .*

Overview. To show the non-existence of computable bounds for length generalization in C-RASP and transformers, we use the fact that non-asymptotic length generalization is equivalent to the decidability of the language equivalence problem for any finite class of languages (Chen et al., 2025, Lemma 3.4, Theorem 3.2). We note that, for C-RASP-definable languages, decidability of language equivalence is equivalent to decidability of nonemptiness, and we prove undecidability of emptiness of C-RASP-definable languages via a reduction from the undecidability of Hilbert’s 10th problem (Hilbert, 1902).

To show exponential bounds for length generalization in C-RASP₊ and the equivalent class of $(1, 1)$ -precision transformers, we show how C-RASP₊ is equivalent to the unary temporal logic TL[$\diamond$] with only the (strict) past operator, and therefore defines the same class of languages as $(1, 1)$ -precision transformers (Li & Cotterell, 2025). This translation into TL[$\diamond$] incurs an exponential blow-up. Then, since each satisfiable formula φ in TL[$\diamond$] has a witnessing string of length polynomial in $|\varphi|$, we obtain an exponential length

generalization bound for C-RASP₊. We also show that this exponential bound is tight in the worst case.

We note that previous work has also investigated the complexity of emptiness checking for different transformer variants (though these results do not imply ours). Sälzer et al. (2025) showed the first undecidability result for log-precision transformer encoders with average-hard attention layers. Later, Sälzer et al. (2026) established undecidability of language emptiness (and therefore of equivalence) for uniform-attention transformers. In general, C-RASP is a more restricted class than all of the above; thus, our undecidability result is stronger than previous results. For the case of unique-hard attention transformers — which are able to express LTL (i.e. order-sensitive) properties (Barceló et al., 2024; Yang et al., 2024) — Bergsträßer et al. (2026) showed that emptiness checking is EXPSPACE-complete.

2. Preliminaries

2.1. Notation

We write $\mathbb{N}$ for the set of natural numbers including 0. We write $[n]$ for the set $\{1, \dots, n\}$. Let Σ be a finite alphabet. The set of all strings over Σ is denoted by Σ^* . If $w \in \Sigma^*$ and $\sigma \in \Sigma$, we write $|w|_\sigma$ for the number of occurrences of σ in w . We write $|w|$ for the length of w and w_i for the symbol at position $i \in [|w|]$. By $\Sigma^{\leq n}$ with $n \in \mathbb{N}$, we mean the set of all strings $w \in \Sigma^*$ of length at most n .

2.2. C-RASP (Counting RASP)

We define the syntax and semantics of C-RASP (akin to the logics of Yang & Chiang (2024); Barceló et al. (2024)). In the sequel, we will define other logics C-RASP₊ and TL[$\Diamond$] as fragments of C-RASP.

Definition 2.1. The syntax of C-RASP formulas is defined:

$$\begin{aligned} \phi &::= \sigma \mid \Diamond \phi \mid \Box \phi \mid \neg \phi_1 \mid \phi_1 \wedge \phi_2 \mid \sum_{t \in \mathcal{T}} \alpha_t t \sim k \\ t &::= \#[\phi_1] \mid c \end{aligned}$$

where $\sigma \in \Sigma$, $\alpha_i, k, c \in \mathbb{Z}$ and $\sim \in \{<, \leq, =, \geq, >\}$. The semantics of formulas is defined as follows:

$$\begin{aligned} w, i &\models \sigma && \iff w_i = \sigma \\ w, i &\models \neg \phi && \iff w, i \not\models \phi \\ w, i &\models \phi_1 \wedge \phi_2 && \iff w, i \models \phi_1 \text{ and } w, i \models \phi_2 \\ w, i &\models \Diamond \phi && \iff w, j \models \phi \text{ for some } j < i \\ w, i &\models \Box \phi && \iff w, j \models \phi \text{ for all } j \leq i \\ w, i &\models \sum_{t \in \mathcal{T}} \alpha_t t \sim k && \iff \sum_{t \in \mathcal{T}} \alpha_t t^{w, i} \sim k. \end{aligned}$$

The semantics of terms is defined as follows:

$$\begin{aligned} \#[\phi]^{w, i} &= |\{j \in [1, i] \mid w, j \models \phi\}| \\ c^{w, i} &= c. \end{aligned}$$

We write $w \models \phi$ if $w, |w| \models \phi$, and we say that ϕ defines the language $\mathcal{L}(\phi) = \{w \mid w \models \phi\}$.

First, $\Diamond \phi$ (“previously”) can be viewed as syntactic sugar for $\#[\phi] \geq 2 \vee (\neg \phi \wedge \#[\phi] \geq 1)$; it is true at position i iff ϕ is true for some position $j < i$. Secondly, $\Box \phi$ (“historically”) can be viewed as syntactic sugar for $\#[\neg \phi] = 0$; it is true at position i iff ϕ is true at all positions $j \leq i$. Finally, $\Box \phi$ is also equivalent to $\phi \wedge \neg(\Diamond \neg \phi)$, noting the strictness of $\Diamond$ and the non-strictness of $\Box$.

In the sequel, we will use a DAG (directed acyclic graph) representation of C-RASP formulas, where a subformula φ may be used *multiple times* in a formula. Such a formula can be thought of as a straight-line *program*, i.e., a sequence $\varphi = (\varphi_i)_{i=1}^n$, where φ_i is any C-RASP definition that could refer to φ_j with $j < i$. The *size* $|\varphi_i|$ of a definition is the number of symbols, where we assume constants to be encoded in binary and each reference to φ_j for $j < i$ to be of size 1. Then the size $|\varphi|$ of φ is defined to be $\sum_{i=1}^n |\varphi_i|$. For example, the formula $\bigwedge_{a \in \Sigma} (\varphi \rightarrow \#a \geq k)$ — which says that if φ is true, then every a occurs in the (non-strict) past at least k times — can be represented by a program of size $O(|\varphi| + |\Sigma| \log(k))$. Note that $|\varphi|$ is counted once (not $|\Sigma|$ times).

Definition 2.2. The *depth* of formulas and terms is defined:

$$\begin{aligned} \text{dp}(\sigma) &= \text{dp}(c) = 0 \\ \text{dp}(\neg \phi) &= \text{dp}(\phi) \\ \text{dp}(\phi_1 \wedge \phi_2) &= \max\{\text{dp}(\phi_1), \text{dp}(\phi_2)\} \\ \text{dp}(\#[\phi]) &= \text{dp}(\Diamond \phi) = \text{dp}(\Box \phi) = \text{dp}(\phi) + 1 \\ \text{dp}\left(\sum_{t \in \mathcal{T}} \alpha_t t \sim k\right) &= \max_{t \in \mathcal{T}} \text{dp}(t). \end{aligned}$$

We write C-RASP_k for the set of all C-RASP formulas with depth at most k .

The *depth* of a C-RASP program P is $\max_{\phi \in P} \text{dp}(\phi)$. The *precision* $\text{prec}(P)$ is the number of bits needed to encode its largest constant. The *girth* $\text{girth}(P)$ is the maximum number of summands occurring in any sum of P .

Example 2.3. First, define D_k , the Dyck language of bounded depth k , as the language of strings with the following regular expression defined inductively:

$$\begin{aligned} D_1 &= (ab)^* \\ D_{k+1} &= (aD_k b)^* \end{aligned}$$

This is the language of balanced parentheses, using a and b instead of $($ and $)$, with a maximum nesting depth of 4. Below is a C-RASP program which recognizes D_4 :

Dyck of depth 4

$$\begin{aligned} \phi_{\text{lower}} &:= \overleftarrow{\#}a - \overleftarrow{\#}b \geq 0 & (1) \\ \phi_{\text{upper}} &:= \overrightarrow{\#}a - \overrightarrow{\#}b \leq 4 & (2) \\ \phi_{\text{bounded}} &:= \phi_{\text{lower}} \wedge \phi_{\text{upper}} & (3) \\ \phi_{\text{all.bounded}} &:= \Box \phi_{\text{bounded}} & (4) \\ \phi_{\text{balanced}} &:= \overleftarrow{\#}a = \overleftarrow{\#}b & (5) \\ \phi_{D_4} &:= \phi_{\text{all.bounded}} \wedge \phi_{\text{balanced}} & (6) \end{aligned}$$

This program has depth 2, girth 2, and precision 2.

2.3. Transformers

Our results concern two variants of transformers in this paper, which round to finite-precision in different ways.

Definition 2.4. A (p, q) -precision transformer is one which uses $O(p)$ bits of precision outside of attention and $O(q)$ bits of precision inside of attention.

Please see [Appendix B.1](#) for precise definitions.

2.4. Computational Learning Theory

We discuss notions from computational learning theory (cf. [Kearns & Vazirani, 1994](#), Chapter 1.2.2) instantiated to formal languages. Our learning algorithm learns a *hypothesis* (a language $S \subseteq \Sigma^*$). Let $\mathcal{H}$ be a set of possible hypotheses that can be learned. A *representation scheme* $\mathcal{L}$ for $\mathcal{H}$ is a surjective partial function from strings in Γ^* to hypotheses in $\mathcal{H}$. If $\mathcal{L}(E) = w$, we say that E *represents* w .

This allows us to measure the *size* of a hypothesis by the length of its shortest representation. The *descriptive complexity* of a hypothesis $L \in \mathcal{H}$ with respect to $\mathcal{L}$ is the length of the shortest representation(s) for L , that is, $\min\{|E| : E \in \Gamma^*, \mathcal{L}(E) = L\}$.

2.5. Non-Asymptotic Length Generalization

Suppose we want to learn a hypothesis L , and we know that L has descriptive complexity (with respect to $\mathcal{L}$) at most n . Up to what string length do we need to see training strings, so that we can learn a representation E with $\mathcal{L}(E) = L$ of size at most n ?

The notion of *length complexity* gives a way to answer this question.

Definition 2.5 (Length complexity). Given a hypothesis class $\mathcal{H}$ and a representation scheme $\mathcal{L}$ for $\mathcal{H}$, the *length complexity* of $\mathcal{H}$ with respect to $\mathcal{L}$ is the minimal function $f_{\mathcal{L}} : \mathbb{N} \rightarrow \mathbb{N}$ such that for any two hypotheses with descrip-

tional complexity (with respect to $\mathcal{L}$) at most c , there is a string of length at most $f_{\mathcal{L}}(c)$ that distinguishes them. That is,

$$f_{\mathcal{L}}(c) = \max_{\substack{E, E' \in \Gamma^{\leq c} \\ \mathcal{L}(E) \setminus \mathcal{L}(E') \neq \emptyset}} \min\{|w| : w \in \mathcal{L}(E) \setminus \mathcal{L}(E')\}.$$

If $f_{\mathcal{L}}(c)$ has a *computable* upper bound on the maximum string length, then it is possible in principle to learn any language $L \in \mathcal{H}$ perfectly, in the following way:

1. Receive the maximum descriptive complexity $c \in \mathbb{N}$.
2. Compute a maximum string length $N \in \mathbb{N}$.
3. Receive training data $T = \{w \in L : |w| \leq N\}$.
4. Output representation $E \in \Gamma^*$ such that $|E| \leq n$, $\mathcal{L}(E) \cap \Sigma^{\leq n} = T$, and $\mathcal{L}(E) = L$.

Step 3 is computable because of our assumption that membership is decidable for $\mathcal{L}$. The learner (step 4) works, in principle, by enumerating all possible hypotheses (as there are only finitely many of them with descriptive complexity at most n) and checking each one against the training data. Assuming that the true hypothesis has descriptive complexity (with respect to $\mathcal{L}$) at most c , length generalization ensures the uniqueness of $\mathcal{L}(E)$. This definition is akin to the problem of finding a minimum representation in computational learning theory (e.g., [Kearns & Vazirani, 1994](#)).

Therefore, whether the length complexity of $\mathcal{H}$ (with respect to $\mathcal{L}$) can be computably bounded (step 2) is an indication of whether learning length generalization is possible. This was shown by [Chen et al. \(2025\)](#) to be equivalent to decidability of language equivalence.

Proposition 2.6 ([Chen et al. 2025](#), Lemma 3.4). *For any hypothesis class $\mathcal{H}$ over formal languages and representation scheme $\mathcal{L} : \Gamma^* \rightarrow \mathcal{H}$ with decidable membership problem, there is a computable upper bound on length complexity for $\mathcal{H}$ with respect to $\mathcal{L}$ iff language equivalence (that is, given $E, E' \in \Gamma^*$, check whether $\mathcal{L}(E) = \mathcal{L}(E')$) is decidable.*

In our setting, $\mathcal{H}$ will always be a set of formal languages $L \subseteq \Sigma^*$. In particular, we take $\mathcal{H}$ to be various subset of formal languages expressible by transformers, i.e., C-RASP-definable languages. We assume a representation scheme $\mathcal{L}$ such that checking $w \in \mathcal{L}(E)$, for any given string $w \in \Sigma^*$, is decidable. In particular, this is the case if we use C-RASP or any equivalent representation of C-RASP, for example, $(1, \log n)$ -precision transformers.

3. Undecidability of C-RASP

Our main result establishes the undecidability of language emptiness for C-RASP, thus answering an open question of Chen et al. (2025).

Theorem 3.1. *It is undecidable whether a given C-RASP program defines the empty language $\emptyset$ or not.*

We prove this by reduction from solvability of Diophantine equations. The decidability of Diophantine equations was Hilbert’s Tenth Problem and answered negatively by Matiyasevich (1993).

Any Diophantine equation can equivalently be expressed as a system of equations of the form $x = c$, $x + y = z$, or $x \cdot y = z$, where $x, y, z \in \mathcal{V}$ are variables ranging over $\mathbb{N}$ and $c \in \mathbb{N}$ is a constant (Matiyasevich, 1993, p. 3). Furthermore, we may assume without loss of generality that in any equation, all the variables are distinct. (If an equation contains x twice, rename one of them to x' and add the equations $x' = x + e$, $e = 0$.)

Given a system of equations over $\mathcal{V}$, let $\Sigma = \mathcal{V} \cup \{\underline{x} : x \in \mathcal{V}\}$. We will construct a C-RASP-definable language $\mathcal{L} \subseteq \Sigma^*$ such that $\mathcal{L} \neq \emptyset$ iff the equations have a solution.

Definition 3.2. We say that a language $\mathcal{L} \subseteq \Sigma^*$ *encodes* an equation $\mathcal{D}$ over $\mathcal{V} \subseteq \Sigma$ if

- If $w \in \mathcal{L}$, then $x_1 \mapsto |w|_{x_1}, \dots, x_m \mapsto |w|_{x_m}$ satisfies $\mathcal{D}$.
- For any $n_1, \dots, n_m \in \mathbb{N}$ satisfying $\mathcal{D}$, there is a string $w \in \mathcal{L}$ such that $|w|_{x_i} = n_i$ for all $i \in [m]$.

Definition 3.3. A formula of C-RASP is *regulated* if no atomic formula σ (for any $\sigma \in \Sigma$) occurs outside of a counting operator $\#$. Let *regulated* C-RASP be the set of all regulated formulas of C-RASP.

First, we will show that equations of the aforementioned forms can be encoded by languages definable in regulated C-RASP. Then we will show how to combine these languages to obtain the result.

3.1. Encoding One Equation

Lemma 3.4. *Any equation $x = c$, $x + y = z$, or $x \cdot y = z$ can be encoded by a language definable in regulated C-RASP.*

Proof. The first two cases are easy; multiplication is more difficult.

For any $A \subseteq \Sigma$, define the following formula, which restricts strings to only use symbols in A :

$$\text{Only}_A := \bigwedge_{\sigma \in \Sigma \setminus A} \# \sigma = 0.$$

Constants An equation $x = c$ is encoded by the language $\mathcal{L} = \{x^c\}$, which is defined by the formula $(\#x = c \wedge \text{Only}_x)$.

Addition An equation $x + y = z$ is encoded by the language $\{w \in \{x, y, z\}^* : |w|_x + |w|_y = |w|_z\}$, which is defined by the formula $(\#x + \#y = \#z \wedge \text{Only}_{x,y,z})$.

Multiplication An equation $x \cdot y = z$ is encoded by the language $\mathcal{L} = \{x^n (z^n y z^n y)^m : m, n \in \mathbb{N}\}$, which encodes $x \cdot y = z$. We need to show that $\mathcal{L}$ is definable in regulated C-RASP.

First, we require that y ’s and $\underline{y}$ ’s strictly alternate, which is equivalent to: (1) at each position, the number of y ’s exceeds the number of $\underline{y}$ ’s by 0 or 1 and (2) at the end of the string, the number of $\underline{y}$ ’s and y ’s are equal.

y and $\underline{y}$ strictly alternating

$$\phi_1 := (\#y = \#y \vee \#y = \#y + 1) \quad (1)$$

$$\phi_2 := (\#y = \#y) \quad (2)$$

$$\text{Alt}_y := \Box \phi_1 \wedge \phi_2 \wedge \text{Only}_{y,\underline{y}} \quad (3)$$

We extend this formula to define $x^*(z^* y z^* y)^*$. We need that (3) no other symbols precede an x , (4) z only occurs when the numbers of y ’s and $\underline{y}$ ’s are equal, and (5) $\underline{z}$ only occurs when the number of y ’s exceeds the number of $\underline{y}$ ’s.

$x^(z^* y z^* y)^*$*

$$\phi_3 := x \rightarrow (\#y = 0 \wedge \#z = 0) \quad (1)$$

$$\phi_4 := z \rightarrow (\#y = \#y) \quad (2)$$

$$\phi_5 := \underline{z} \rightarrow (\#y = \#y + 1) \quad (3)$$

$$\phi_6 := \Box \phi_3 \wedge \Box \phi_4 \wedge \Box \phi_5 \quad (4)$$

$$\text{Ord}_{x,y,z} := \text{Alt}_y \wedge \phi_6 \wedge \text{Only}_{x,y,z,\underline{y},\underline{z}} \quad (5)$$

Now to define $x^n (z^n y z^n y)^m$, we need the additional constraints that all blocks of x, z , and $\underline{z}$ have the same size. This can be achieved by asserting that (6) at every y , the difference between $\#z$ and $\#\underline{z}$ is equal to $\#x$, and (7) at every $\underline{y}$, they are balanced.

$$x^n(z^n y \underline{z}^n y)^m$$

$$\phi_7 := y \rightarrow (\#x = \#z - \#z) \quad (1)$$

$$\phi_8 := y \rightarrow (\#z = \#z) \quad (2)$$

$$\phi_{x \cdot y = z} := \text{Ord}_{x,y,z} \wedge \exists \phi_7 \wedge \exists \phi_8 \quad (3)$$

□

3.2. Combining Equations

First, we show how to concatenate multiple languages that encode equations.

Lemma 3.5. *Let Σ be an alphabet and $\$ \notin \Sigma$. For any languages $\mathcal{L}_1, \dots, \mathcal{L}_n$ over Σ such that each $\mathcal{L}_i$ for $i \in [n]$ is definable in regulated C-RASP, the language $\mathcal{L}_1 \$ \mathcal{L}_2 \$ \dots \$ \mathcal{L}_n$ is definable in C-RASP.*

Proof. For any $i \in [n]$, let ϕ_i be a regulated formula defining $\mathcal{L}_i$. Let ϕ'_i be the result of taking ϕ_i and replacing every subformula $\# \phi$ with $\#((\# \$ = i - 1) \wedge \phi)$. Observe that $\phi'_i \wedge (\# \$ = n)$ defines the language $\{u \$ v \$ w : |u|_{\$} = i - 2, v \in \mathcal{L}_i, |w|_{\$} = n - i\}$. Then construct the formula

$$\phi_{\mathcal{L}} := \phi'_1 \wedge \dots \wedge \phi'_n \wedge (\# \$ = n). \quad \square$$

This allows us to reduce the solving of Diophantine equations to emptiness-checking in C-RASP.

Proof of Theorem 3.1. Consider any Diophantine equation $\mathcal{D}$ with $\mathcal{V}$ as the set of all variables. We can rewrite it as a system of equations $\mathcal{D}_i$ of the form $x = c$, $x + y = z$, and $x \cdot y = z$, where x, y, z are variables ranging over $\mathbb{N}$ and $c \in \mathbb{N}$ is a constant. For each $\mathcal{D}_i$, by Lemma 3.4 there is a C-RASP-definable language $\mathcal{L}_i$ that encodes $\mathcal{D}_i$. By Lemma 3.5 there is a C-RASP formula $\phi_{\mathcal{L}}$ defining $\mathcal{L}_1 \$ \mathcal{L}_2 \$ \dots \$ \mathcal{L}_n$. It remains to add additional constraints on ϕ to ensure that each $\mathcal{L}_i$ has the same number of occurrences of each variable.

We add for each variable x the following conjunction:

$$\psi_x := \bigwedge_{i=1}^{n-1} \left(\# [x \wedge \# \$ = i] = \# [x \wedge \# \$ = i + 1] \right).$$

Then we define the following formula

$$\phi := \phi_{\mathcal{L}} \wedge \bigvee_{x \in \mathcal{V}} \psi_x.$$

Finally, we can see that $\mathcal{L}(\phi)$ is nonempty iff $\mathcal{D}$ has a solution. □

Corollary 3.6. *The length complexity of C-RASP is not computably bounded.*

Proof. By Lemma 4.5 of Chen et al. (2025), the length complexity is computably bounded iff language equivalence is decidable, which is decidable iff language emptiness is decidable. The result follows from Theorem 3.1. □

This applies even to C-RASP programs of depth 2, as the formulas discussed above are all of depth 2. We remark that in general, C-RASP can be reduced to C-RASP with two layers while preserving language nonemptiness. This is shown in Appendix C.

3.3. Relationship to Previous Bounds

We have just shown that the length complexity of C-RASP₂ is uncomputable, which appears to contradict the bound shown by Chen et al. (2025). However, this is because they consider a restricted subset of programs, which they notate as C-RASP^{2,K,T}. In essence, their restricted C-RASP lacks constant bias terms (e.g. $\#a - \#b \leq 4$). We prove this class is strictly weaker than C-RASP₂ (even with bounds on size, precision, and girth).

Theorem 3.7. *Dyck languages of bounded depth are not expressible in C-RASP^{2,K,T}, for any K and T .*

On the other hand, bounded-depth Dyck languages are straightforwardly expressed by C-RASP₂ with bounded size, like in Example 2.3. The full definition of C-RASP^{2,K,T} and a proof can be found in Appendix A.1.

Similarly, Izzo et al. (2026) also derive exponential bounds for a specific theoretical definition of a two layer transformer. This specific definition is based off of the limit transformer defined by Huang et al. (2025) (which can simulate C-RASP programs) except with Lipschitz-continuity enforced in the position-wise functions. Since non-Lipschitz-continuity appears essential for simulation of C-RASP programs (Yang & Chiang, 2024; Huang et al., 2025), this suggests that their model is not comparable with C-RASP, and thus also does not contradict our results. More details can be found in Appendix A.2.

4. Positive Fragment of C-RASP

While the general class of C-RASP programs has an undecidable language emptiness problem, and thus uncomputable length complexity, we find that a natural subset of C-RASP does in fact admit computable length complexity bounds. In essence, this restricts C-RASP so that one can only count up to a threshold. The logic can be seen as a version of counting LTL (Laroussinie et al., 2010) while omitting the **until** operator.

Definition 4.1. The syntax of C-RASP_+ is defined:

$$\begin{aligned}\phi &::= \sigma \mid \neg\phi_1 \mid \phi_1 \wedge \phi_2 \mid \sum_{t \in \mathcal{T}} \alpha_t t \sim k \\ t &::= \#[\phi_1] \mid c\end{aligned}$$

where $\alpha_t, k, c \in \mathbb{N}$ and $\sim \in \{<, \leq, =, \geq, >\}$. See [Definition 2.1](#) for the semantics.

A C-RASP_+ program is said to be *decomposed* if all counting formulas are of the form $\#[\phi] \geq c$, where ϕ is decomposed and $c \in \mathbb{N}$. Note that the girth of a decomposed program is at most 1.

Lemma 4.2. *For every C-RASP_+ program of size n , depth d , precision p , and girth g there is an equivalent decomposed C-RASP_+ program of size $O(n 2^{\text{poly}(p,g,d)})$, depth $O(d)$, and precision $O(p)$.*

Proof. First, note that we can rewrite

$$\begin{aligned}\sum_{i=1}^k \alpha_i t_i > c &\quad \text{to} \quad \sum_{i=1}^k \alpha_i t_i \geq c + 1 \\ \sum_{i=1}^k \alpha_i t_i = c &\quad \text{to} \quad \sum_{i=1}^k \alpha_i t_i \geq c \wedge \neg \left(\sum_{i=1}^k \alpha_i t_i \geq c + 1 \right) \\ \sum_{i=1}^k \alpha_i t_i \leq c &\quad \text{to} \quad \neg \left(\sum_{i=1}^k \alpha_i t_i \geq c + 1 \right) \\ \sum_{i=1}^k \alpha_i t_i < c &\quad \text{to} \quad \neg \left(\sum_{i=1}^k \alpha_i t_i \geq c \right).\end{aligned}$$

In any counting formula $\sum_{i=1}^k \alpha_i t_i \geq c$, since we can remove any summand $0 \cdot t_i$, we may assume without loss of generality that $\alpha_i > 0$ for all i . Let

$$S := \{(c_1, \dots, c_k) \in [0, c]^k : \alpha_1 c_1 + \dots + \alpha_k c_k \geq c\}.$$

We now write $\sum_{i=1}^k \alpha_i t_i \geq c$ as $\bigvee_{(c_1, \dots, c_k) \in S} \bigwedge_{i=1}^k t_i \geq c_i$ and replace every $t_i \geq c_i$ where t_i is a constant with $\top$ or $\perp$ depending on whether the inequality is satisfied. Note that the size of S is bounded by $(c+1)^k$, i.e., exponential in the precision p and girth g . Thus, the size of the resulting program is $O(n 2^{\text{poly}(p,g,d)})$. $\square$

Definition 4.3. The syntax of $\text{TL}[\Diamond]$ is defined:

$$\phi ::= \sigma \mid \neg\phi_1 \mid \phi_1 \wedge \phi_2 \mid \Diamond\phi \mid \Box\phi$$

See [Definition 2.1](#) for the semantics.

Proposition 4.4. *For every C-RASP_+ program of size n , depth d , precision p , and girth g there exists an equivalent $\text{TL}[\Diamond]$ program of size $O(n 2^{\text{poly}(p,g,d)})$.*

Proof. For the translation from C-RASP_+ to $\text{TL}[\Diamond]$ we use [Lemma 4.2](#) to assume a decomposed C-RASP_+ program P . Since atomic formulas and Boolean operations can be directly expressed, it suffices to consider formulas of the form $\#[\phi] \geq c$, where by induction we assume that we already defined a $\text{TL}[\Diamond]$ formula ψ such that $w, i \models \psi$ iff $w, i \models \phi$ for all $w \in \Sigma^*$ and $i \in [|w|]$. We then define

$$\begin{aligned}\varphi &:= \Diamond(\psi \wedge \Diamond(\psi \wedge \dots \Diamond(\psi \wedge \Diamond\psi) \dots)) \vee \\ &\quad \psi \wedge \Diamond(\psi \wedge \Diamond(\psi \wedge \dots \Diamond(\psi \wedge \Diamond\psi) \dots))\end{aligned}$$

where the $\Diamond$ nesting depth in the first row is equal to c and in the second row equal to $c - 1$. (But if $c = 0$, we set $\varphi := \top$.) The second row is needed since $\#[\phi]$ includes the current position in the counting, whereas $\Diamond\psi$ is interpreted as strictly in the past. Note that $|\varphi| \in O(c \cdot |\psi|)$, that is, exponential in the number of bits needed to encode c . Thus, together with [Lemma 4.2](#) this yields a $\text{TL}[\Diamond]$ program of size $O(n 2^{\text{poly}(p,g,d)})$ in total. $\square$

Lemma 4.5. *For every $\text{TL}[\Diamond]$ program P with $\mathcal{L}(P) \neq \emptyset$ there exists a string $w \in \mathcal{L}(P)$ of length at most polynomial in the size of P .*

Proof. Let P be the $\text{TL}[\Diamond]$ program $P_1 := \phi_1, \dots, P_k := \phi_k$ over alphabet Σ . We first construct a single formula (not in DAG representation)

$$\psi := \Box((P_1 \leftrightarrow \phi_1) \wedge \dots \wedge (P_{k-1} \leftrightarrow \phi_{k-1})) \wedge \phi_k$$

where we slightly adapt the semantics so that it accepts strings of sets of propositions $\Gamma := \Sigma \cup \{P_1, \dots, P_k\}$. More precisely, $w', i \models \gamma$ iff $\gamma \in w'_i$ for $\gamma \in \Gamma$, $w' \in (2^\Gamma)^*$, and $i \in [|w'|]$. It is shown in ([Etessami et al., 2002](#)) that $\text{TL}[\Diamond]$ formulas with this adapted semantics have the property that if the accepted language is nonempty, then there is an accepted string of length at most polynomial in the size of the formula. That is, under the assumption that the language of ψ is nonempty, there is a string $w' \in (2^\Gamma)^*$ whose length is at most polynomial in the size of ψ . Observe that by definition of ψ , there is a word $w \in \Sigma^*$ such that $w_i \in w'_i$ for all i . Thus, w is a word accepted by P of length at most polynomial in the size of P . $\square$

We use this lemma to prove length complexity bounds.

Proposition 4.6. *For every C-RASP_+ program P with $\mathcal{L}(P) \neq \emptyset$ there exists a string $w \in \mathcal{L}(P)$ of length at most exponential in the size of P . This bound is in fact optimal in the worst case.*

Proof. Given a C-RASP_+ program P with $\mathcal{L}(P) \neq \emptyset$, we first apply [Proposition 4.4](#) to obtain a $\text{TL}[\Diamond]$ program P' with $\mathcal{L}(P') = \mathcal{L}(P)$ of size at most exponential in the size

of P . By Lemma 4.5, if $\mathcal{L}(P') \neq \emptyset$, there exists a string of length at most polynomial in the size of P' , i.e., exponential in the size of P , as required.

To see that the exponential bound is optimal in the worst case, consider the infinite family $\{P_n\}_{n \geq 1}$ of C-RASP₊ programs $P_n := (\#[a] = n)$ over the alphabet $\Sigma = \{a\}$, where the number n is encoded in binary. We observe that the smallest (and only) string contained in $\mathcal{L}(P_n)$ has length n , which is exponential in the number of bits needed to encode n , so also exponential in the size of P_n . $\square$

Theorem 4.7. *The length complexity of a C-RASP₊ program is exponential in the size of the program.*

Proof. Given C-RASP₊ programs P and P' , the C-RASP₊ program $D := (P \wedge \neg P') \vee (\neg P \wedge P')$ recognizes exactly the set of strings that distinguish P and P' . By Proposition 4.6, the smallest string in $\mathcal{L}(D)$, if nonempty, is of length at most exponential in the size of D . Thus, if $\mathcal{L}(P) \neq \mathcal{L}(P')$, there exists a distinguishing string of length at most exponential in the size of P and P' . $\square$

5. Implications for Transformers

The previous sections discussed length complexity results for C-RASP and C-RASP₊, which we connect to transformers here. We consider transformers that round numbers to a fixed number of bits of precision, but in two slightly different ways. A $(1, \log n)$ -precision transformer does not round inside self-attention, while a $(1, 1)$ -precision transformer rounds even inside self-attention. Intuitively, the distinction boils down to being able to attend uniformly to every single position or only being able to attend to a fixed number of positions. Please see Appendix B.1 for precise definitions.

For the first main result, because $(1, \log n)$ -precision transformers and C-RASP programs define the exact same class of languages (cf. Yang et al., 2025, Theorem 3.1), the uncomputable length complexity bound from Section 3 applies directly to these transformers.

Theorem 5.1. *$(1, \log n)$ -precision Transformers (even with two layers) do not admit non-asymptotic length generalization.*

This theorem implies that no learning algorithm can decide if a $(1, \log n)$ -precision transformer has seen enough data. The same bound applies to any class of transformers which are known to contain C-RASP. For instance, transformers with polynomial padding tokens, which subsume FO-uniform TC⁰ (Merrill & Sabharwal, 2025), transformers with polynomial temperature scaling (Yang et al., 2026a), and limit transformers (Huang et al., 2025).

In contrast, by bounding the precision within attention, we

find that $(1, 1)$ -precision transformers do admit length generalization.

Theorem 5.2. *$(1, 1)$ -precision transformers admit length generalization with exponential length complexity.*

This is shown using the expressive equivalence between $(1, 1)$ -precision transformers and C-RASP₊, which is a corollary of the equivalence between $(1, 1)$ -precision transformers and TL[$\diamond$] (cf. Li & Cotterell, 2025), and Proposition 4.4. We provide a direct translation into C-RASP₊ in Appendix B, and derive upper bounds on its size, depth, precision, and girth.

First, we show a singly exponential length complexity lower bound for $(1, 1)$ -precision transformers via a polynomial sized translation from C-RASP₊ to $(1, 1)$ -precision transformers.

Proposition 5.3. *The length complexity of $(1, 1)$ -precision transformers with precision p , dimension d , and depth L is $\Omega(2^{\text{poly}(p, d, L)})$.*

Proof. The length complexity of C-RASP₊ is exponential (Theorem 4.7). By Theorem B.11 there is a polynomial transformation from C-RASP₊ programs into transformers, attaining the bound. $\square$

Intuitively, this means that any learning algorithm would need to check strings of exponential length before being able to identify the ground-truth solution which is able to length generalize. Consider the witnessing language $\# [a] \geq 2^p$ (all strings with at least 2^p a 's). A 1-layer $(1, 1)$ -precision transformer with precision p can recognize this language, and this transformer only accepts strings of exponential length $\geq 2^p$.

For the matching singly exponential upper bound, the translation shown in Appendix B.3 produces a C-RASP₊ program with exponential size, but linear depth, precision, and girth. Thus, the translation into TL[$\diamond$] shown in Proposition 4.4 does not incur an additional blowup, and the entire program will have exponential size.

Proposition 5.4. *$(1, 1)$ -precision transformers can be converted to exponentially large equivalent TL[$\diamond$] formulas.*

Whether or not there exists a polynomial size translation from $(1, 1)$ -precision transformers into C-RASP₊, we leave as an open question. Then, using this proposition we can derive the upper bound.

Proposition 5.5. *The length complexity of $(1, 1)$ -precision transformers with precision p , dimension d , and depth L is $O(2^{\text{poly}(p, d, L)})$.*

Proof. By the previous proposition, we can find an equivalent TL[$\diamond$] program of exponential size. By Lemma 4.5, we

only need to check strings of length polynomial in the program size, and thus exponential in the transformer size. $\square$

6. Discussion

We have shown that there is no algorithm for learning length generalizing transformers in the general case, and any algorithm for $(1, 1)$ -precision transformers must train on strings of exponential length. This was shown by analyzing the complexity of the language emptiness problem for C-RASP and C-RASP₊, in order to show that length complexity was uncomputable and exponential, respectively.

Our results provide a perspective on the observed difficulty of length generalization in transformers. First, length generalization in practice is often quite sensitive to the initialization of the model, learning rate, and other intricacies (Zhou et al., 2024b). Furthermore, even in controlled experiments, length generalization is only ever observed to be partial. For instance, transformers may exhibit generalization from lengths 50 to 150 (Huang et al., 2025), or even 40 to 500 (Li & Cotterell, 2025; Delétang et al., 2023), but performance inevitably degrades at long enough lengths. In these experiments, these transformers have enough depth, width, and precision to express the ground truth solution, so any deficiencies must be as a result of learning dynamics. Our results give one reason for these failures: any learning algorithm may need to see unfeasibly long strings in order to learn a perfectly length-generalizing transformer.

We have seen that length-generalization as an empirical phenomenon does not follow the typical scaling laws in machine learning, and logical complexity provides a useful perspective. In particular, scaling model size and data does not necessarily help, while expressivity in C-RASP accurately predicts length generalization (Huang et al., 2025). This insight is not novel, but our bounds bring this insight closer to practice by proving the first length-complexity bounds for entire classes of transformers, rather than restricted subclasses.

An important future research direction is to identify other fragments of C-RASP and limit transformers, for which computable length generalization bounds exist. In addition, we believe that a more fine-grained analysis of the sample size required for length generalization — although it has not been much addressed for transformer languages — is important. For example, although C-RASP₊ has an exponential length generalization bound, it is possible that the required sample size is still of a polynomial size. We leave this as future work.

Impact Statement

This paper presents fundamental results in machine learning. Although there may be potential societal consequences of our work, none are direct enough to be specifically highlighted here.

Acknowledgements

We thank Aarohi Srivastava and Katsumi Ibaraki for helpful feedback on writing, as well as Dana Angluin and Michael Hahn for fruitful discussion. We thank the anonymous reviewers for their helpful comments. This material is based in part upon work supported by Deutsche Forschungsgemeinschaft (grant number 522843867), the European Union  (ERC, LASD, 101089343, <https://doi.org/10.3030/101089343> and FINABIS, 101077902, <https://doi.org/10.3030/101077902>) and the US National Science Foundation (grant number 2502292). Andy Yang is supported by the US National Science Foundation Graduate Research Fellowship Program under Grant No. 2236418.

Views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

References

- Angluin, D. Queries and concept learning. *Machine Learning*, 2(4):319–342, 1988. doi:[10.1007/BF00116828](https://doi.org/10.1007/BF00116828).
- Anil, C., Wu, Y., Andreassen, A. J., Lewkowycz, A., Misra, V., Ramasesh, V. V., Slone, A., Gur-Ari, G., Dyer, E., and Neyshabur, B. Exploring length generalization in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://openreview.net/forum?id=zSkYVeX7bC4>.
- Barceló, P., Kozachinskiy, A., Lin, A. W., and Podolskii, V. V. Logical languages accepted by transformer encoders with hard attention. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=gbrHZq07mq>.
- Bergsträßer, P., Cotterell, R., and Lin, A. W. Transformers are inherently succinct. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=Yxz92UuPLQ>. arXiv:2510.19315.
- Chen, T., Ma, T., and Li, Z. Non-asymptotic length generalization. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*,

2025. URL <https://proceedings.mlr.press/v267/chen25ar.html>.
- Delétang, G., Ruoss, A., Grau-Moya, J., Genewein, T., Wenliang, L. K., Catt, E., Cundy, C., Hutter, M., Legg, S., Veness, J., and Ortega, P. A. Neural networks and the Chomsky hierarchy. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=WbxHAzkeQcn>.
- Etessami, K., Vardi, M. Y., and Wilke, T. First-order logic with two variables and unary temporal logic. *Information and Computation*, 179(2):279–295, 2002. doi:[10.1006/inco.2001.2953](https://doi.org/10.1006/inco.2001.2953).
- Gold, E. M. Language identification in the limit. *Information and Control*, 10(5):447–474, 1967. doi:[10.1016/S0019-9958\(67\)91165-5](https://doi.org/10.1016/S0019-9958(67)91165-5).
- Gold, E. M. Complexity of automaton identification from given data. *Information and Control*, 37(3):302–320, 1978. doi:[10.1016/S0019-9958\(78\)90562-4](https://doi.org/10.1016/S0019-9958(78)90562-4).
- Grädel, E., Kolaitis, P. G., and Vardi, M. Y. On the decision problem for two-variable first-order logic. *Bulletin of Symbolic Logic*, 3(1):53–69, 1997. doi:[10.2307/421196](https://doi.org/10.2307/421196).
- Hilbert, D. Mathematical Problems. *Bulletin of the American Mathematical Society*, 8(10):437–479, 1902. doi:[10.1090/S0002-9904-1902-00923-3](https://doi.org/10.1090/S0002-9904-1902-00923-3).
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J. W., and Sifre, L. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://openreview.net/forum?id=iBBcRUlOAPR>.
- Huang, X., Yang, A., Bhattamishra, S., Sarrof, Y., Krebs, A., Zhou, H., Nakkiran, P., and Hahn, M. A formal framework for understanding length generalization in transformers. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=U49N5V51rU>.
- Izzo, Z., Nichani, E., and Lee, J. D. Quantitative bounds for length generalization in transformers. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=TLSUIyBifs>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Kearns, M. J. and Vazirani, U. *An Introduction to Computational Learning Theory*. The MIT Press, 08 1994. ISBN 9780262276863. doi:[10.7551/mitpress/3897.001.0001](https://doi.org/10.7551/mitpress/3897.001.0001).
- Laroussinie, F., Meyer, A., and Pettonnet, E. Counting LTL. In *Proceedings of the 17th International Symposium on Temporal Representation and Reasoning*, pp. 51–58, 2010. doi:[10.1109/TIME.2010.20](https://doi.org/10.1109/TIME.2010.20).
- Li, J. and Cotterell, R. Characterizing the expressivity of fixed-precision transformer language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://openreview.net/forum?id=29LwAgLFpj>.
- Matiyasevich, Y. V. *Hilbert’s 10th Problem*. Foundations of Computing. MIT Press, Cambridge, MA, 1993. ISBN 9780262132954. URL <https://mitpress.mit.edu/9780262132954/hilberts-10th-problem/>.
- Merrill, W. and Sabharwal, A. A logic for expressing log-precision transformers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://openreview.net/forum?id=uR8TtWCIsr>.
- Merrill, W. and Sabharwal, A. Exact expressive power of transformers with padding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://openreview.net/forum?id=0labxStFcy>.
- Nogueira, R., Jiang, Z., and Lin, J. Investigating the limitations of transformers with simple arithmetic tasks. In *Proceedings of Math-AI: ICLR Workshop on the Role of Mathematical Reasoning in General Artificial Intelligence*, 2021. URL <https://arxiv.org/abs/2102.13019>.
- Nye, M., Andreassen, A. J., Gur-Ari, G., Michalewski, H., Austin, J., Bieber, D., Dohan, D., Lewkowycz, A., Bosma, M., Luan, D., Sutton, C., and Odena, A. Show your work: Scratchpads for intermediate computation with language models. In *Proceedings of the Deep Learning for Code Workshop*, 2022. URL <https://openreview.net/forum?id=HB1x2idbkbq>.
- Sälzer, M., Alsmann, E., and Lange, M. Transformer encoder satisfiability: Complexity and impact on formal reasoning. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*,

2025. URL <https://openreview.net/forum?id=VVO3ApdMUE>.
- Sälzer, M., Köcher, C., Kozachinskiy, A., Zetzsche, G., and Lin, A. W. The counting power of transformers. In *Proceedings of the 14th International Conference on Learning Representations (ICLR)*, 2026. URL <https://openreview.net/forum?id=IAFWK6NyrP>.
- Strobl, L., Merrill, W., Weiss, G., Chiang, D., and Angluin, D. What formal languages can transformers express? A survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 2024. doi:10.1162/TACL_A_00663.
- Varre, A., Yüce, G., and Flammarion, N. Learning in-context n -grams with transformers: Sub- n -grams are near-stationary points. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. URL <https://proceedings.mlr.press/v267/varre25a.html>.
- Wang, Z., Nichani, E., Bietti, A., Damian, A., Hsu, D., Lee, J. D., and Wu, D. Learning compositional functions with transformers from easy-to-hard data. In *Proceedings of the 38th Conference on Learning Theory (COLT)*, pp. 5632–5711, 2025. URL <https://proceedings.mlr.press/v291/wang25a.html>.
- Weiss, G., Goldberg, Y., and Yahav, E. Thinking like transformers. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pp. 11080–11090, 2021. URL <https://proceedings.mlr.press/v139/weiss21a.html>.
- Yang, A. and Chiang, D. Counting like transformers: Compiling temporal counting logic into softmax transformers. In *Proceedings of the First Conference on Language Modeling (CoLM)*, 2024. URL <https://openreview.net/forum?id=FmhPg4UJ9K>.
- Yang, A., Chiang, D., and Angluin, D. Masked hard-attention transformers recognize exactly the star-free languages. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://openreview.net/forum?id=FBMsBdH0yz>.
- Yang, A., Cadilhac, M., and Chiang, D. Knee-deep in C-RASP: A transformer depth hierarchy. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL <https://openreview.net/forum?id=jPduiyxyfw>.
- Yang, A., Strobl, L., Chiang, D., and Angluin, D. Simulating hard attention using soft attention. *Transactions of the Association for Computational Linguistics*, 2026a. doi:10.1162/TACL.a.597.
- Yang, A., Watson, C., Xue, A., Bhattamishra, S., Llaena, J., Merrill, W., Ferreira, E. D. S., Svete, A., and Chiang, D. The transformer cookbook. *Transactions on Machine Learning Research*, 2026b. URL <https://openreview.net/forum?id=sPshCSvDrX>.
- Zhou, H., Bradley, A., Littwin, E., Razin, N., Saremi, O., Susskind, J. M., Bengio, S., and Nakkiran, P. What algorithms can transformers learn? A study in length generalization. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024a. URL <https://openreview.net/forum?id=AssIuHnmHX>.
- Zhou, Y., Alon, U., Chen, X., Wang, X., Agarwal, R., and Zhou, D. Transformers can achieve length generalization but not robustly. In *Proceedings of the ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024b. URL <https://openreview.net/forum?id=DWkWIh3vFJ>.

A. Relationship to Previous Length Complexity Bounds

A.1. Chen et al. (2025)

While Chen et al.'s $\text{C-RASP}^{2,K,T}$ has a length complexity of $T^{O(K)}$, we show that C-RASP_2 has uncomputable length complexity. Here, we show why this is not a contradiction. We will show that $\text{C-RASP}^{2,K,T}$ is strictly distinct from the full class of depth 2 programs C-RASP_2 . The subset $\text{C-RASP}^{2,K,T}$ contains C-RASP programs of the following form:

Definition A.1. (Chen et al., 2025, Definition 5.4) The class $\text{C-RASP}^{2,K,T}$ contains programs over alphabet $\Sigma = \{a, b\}$ parameterized by $0 \leq z \leq T$ and distinct $\alpha_\psi, \beta_\psi, \lambda_\psi \in [-T, T]$ for $\psi \in \Psi = \{\psi_1, \psi_2, \dots, \psi_K\}$, where $\frac{\alpha_\psi}{\beta_\psi} \in (0, 1)$, $1 \leq K \leq T^2$, and $\sum_{\psi \in \Psi} \lambda_\psi > z$, and which implement the formula

$$\phi := \sum_{\psi \in \Psi} \lambda_\psi \# \psi > z \# \top$$

where each ψ is of the form

$$\psi := \alpha_\psi \# a > \beta_\psi \# \top.$$

First, we define Dyck languages of bounded depth, which will be the separating language class:

Definition A.2. Define D_k , the Dyck language of bounded depth k , as the language of strings of the following form:

$$\begin{aligned} D_1 &= (ab)^* \\ D_{k+1} &= (aD_k b)^* \end{aligned}$$

In other words, strings in D_k satisfy $0 \leq \#a - \#b \leq k$ everywhere, and satisfy $\#a = \#b$ at the end of the string.

We prove that bounded Dyck languages are not expressible in $\text{C-RASP}^{2,K,T}$ using a geometric argument. Intuitively, we show that all formulas of $\text{C-RASP}^{2,K,T}$ will eventually output a constant value for enough strings from any D_k for every k . Then since D_{k+1} contains both strings in D_k and outside of D_k , this shows that no such program can recognize any D_k .

First we state a lemma, from which the theorem follows.

Lemma A.3. Let Ψ be a set of C-RASP programs ψ of the form $\alpha_\psi \# a > \beta_\psi \# \top$. There exists a constant $\epsilon_{k,\Psi}$ such that for all $\psi \in \Psi$ and all $w \in D_k$, either $(\# \psi)^{w,i} = i \pm \epsilon_{k,\Psi}$ for all i , or $(\# \psi)^{w,i} = \pm \epsilon_{k,\Psi}$ for all i .

Proof. This can be seen through a geometric interpretation of C-RASP formulas. We can plot the number of a 's and b 's in each word on a 2d grid, where the x -axis denotes $\#a$'s and the y -axis denotes $\#b$. Every string can be viewed as a path through the grid, where steps to the right denote a 's and steps up denote b 's.

Then, each ϕ defines a line ℓ_ϕ demarcating a half-plane, such that the path of a word lands in a half-plane iff that word satisfies ϕ . Visually, we can think of D_k as the set of strings contained within the rectangular band coming out of the origin with width k . That is, the set of strings which correspond to paths that start and end on the line $\#a = \#b$ without ever leaving the band.

Let ℓ_1 be the line with least slope greater than 1 (or the y -axis), and let ℓ_2 be the line with greatest slope less than or equal to 1 (or the x -axis). Figure 1 visualizes D_k , ℓ_1 , and ℓ_2 .

Because ℓ_1 and ℓ_2 are distinct lines coming out of the origin, the distance between them increases without bound the farther we get from the origin. Then there is some length $\epsilon_{k,\Psi}$ such that any strings in D_k with length greater than $\epsilon_{k,\Psi}$ will have paths that do not cross ℓ_1 or ℓ_2 . The truth values of each $\psi \in \Psi$ will then be fixed to a constant $\top$ or $\perp$ for all such strings. $\square$

Theorem A.4. For all k , D_k is not expressible in $\text{C-RASP}^{2,K,T}$ for any K, T .

Proof. Assume for sake of contradiction that ϕ is a $\text{C-RASP}^{2,K,T}$ program which recognizes D_k . Let Ψ be the set of all ψ in ϕ . By Lemma A.3, we can partition Ψ into the set of formulas $\Psi_\top$ (which are satisfied at $|w| \pm \epsilon_{k,\Psi}$ positions) and $\Psi_\perp$

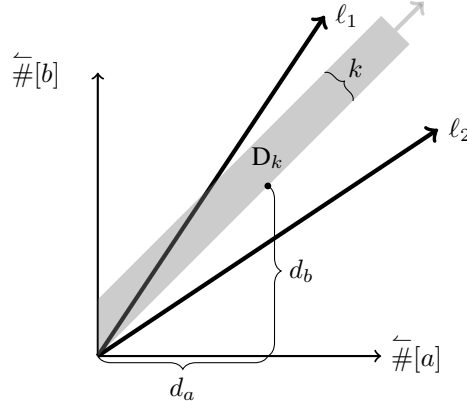


Figure 1. A visualization of the proof idea. ℓ_1 and ℓ_2 are the two lines above and below the line $\#a = \#b$, the horizontal band contains all strings in D_k , and $\epsilon_{k,\Psi} = d_a + d_b$.

(which are satisfied at $\pm\epsilon_{k,\Psi}$ positions) over strings in D_{k+1} . We can plug these values into the definition of ϕ to find that the truth value of ϕ is equivalent to:

$$\sum_{i: \psi \in \Psi_{\top}} \lambda_{\psi}(|w| \pm \epsilon_{k,\Psi}) + \sum_{i: \psi \in \Psi_{\perp}} \lambda_{\psi}(\pm \epsilon_{k,\Psi}) > z|w|.$$

Here, the truth value of ϕ only depends on the length of the string $w \in D_k$ on which it is evaluated. Further grouping terms of the above equation into constants, we can see that the entire formula boils down to the following expression depending only on the length of the string:

$$C_1|w| + C_2 > z|w|.$$

For w with length greater than some sufficiently large threshold, $C_1|w|$ and $z|w|$ vastly outweigh C_2 , and thus the truth value of ϕ becomes constant for any other string beyond the threshold length. This implies either $D_{k+1} \subseteq \mathcal{L}(\phi)$ or $D_{k+1} \subseteq \Sigma^* \setminus \mathcal{L}(\phi)$, restricted to strings of length beyond some threshold. However, since $\mathcal{L}(\phi) = D_k$, this implies that either $D_{k+1} \subseteq D_k$ or $D_{k+1} \subseteq \Sigma^* \setminus D_k$, beyond some length threshold. This contradicts the fact that $D_k \subsetneq D_{k+1}$. Thus, no such program ϕ exists. $\square$

As a final note, we observe that [Chen et al. \(2025\)](#) use the absolute size of the constants T as precision, while we use the bit-precision p of the constants (exponentially smaller than the absolute size). This accounts for why it is non-contradictory that the depth 1 C-RASP fragment they study has quadratic length complexity, while we show an exponential lower bound for C-RASP of depth 1.

A.2. Izzo et al. (2026)

[Izzo et al. \(2026\)](#) prove quantitative length generalization bounds for two variants of transformers. First, they consider transformers in which:

- Parameters and activations are stored with $(1, 1)$ -precision.
- There is only 1 layer.
- Absolute position encodings (APEs) are periodic.
- Relative position encodings (RPEs) are added to the attention logits; they must be local, that is, zero when the query and key distance exceeds a maximum distance.
- The gap between the maximal attention logit and any non-maximal attention logit is guaranteed to be at least γ .
- Attention logits are scaled by $\log n$ where n is the input length.

- Position-wise functions are 1-Lipschitz continuous.

Then they obtain a length generalization bound exponential in $2^{1/\gamma}$ and polynomial in various properties of the transformer. Our [Theorem 5.2](#) gives a similar exponential bound for $(1, 1)$ -precision transformers, but using a more standard definition of transformer and for an arbitrary number of layers.

Second, [Izzo et al. \(2026\)](#) consider transformers in which:

- Parameters and activations are stored with (∞, ∞) -precision.
- There are at most 2 layers.
- Periodic APEs and local RPEs as above.
- The gap between the maximal RPE and any non-maximal RPE is guaranteed to be at least γ .
- The RPEs are scaled by $\log i$ where i is the query position.
- Position-wise functions are 1-Lipschitz continuous.

Then they obtain a length generalization bound exponential in various properties of the transformer. In the case where the absolute and relative position encodings are set to zero, this is a standard transformer, so their bound would appear to contradict our [Theorem 5.1](#). The difference seems to be in their assumption of Lipschitz-continuous position-wise functions. Our [Theorem 5.1](#) is based on a translation of C-RASP to transformers, and all three such translations that we are aware of depend on some non-Lipschitz position-wise function: layer normalization ([Yang & Chiang, 2024](#)), the Heaviside step activation function ([Huang et al., 2025](#)), or rounding ([Yang et al., 2025](#)). Such a function seems critical for testing the equalities in system of a Diophantine equations.

B. Transformer Size Bounds

B.1. Definitions

Definition B.1. A *fixed-precision number* with p total bits and s fractional bits is a rational number of the form $m \cdot 2^{-s}$ where m is an integer and $-2^{p-1} \leq m < 2^{p-1}$. We write $\mathbb{F}_{p,s}$, or simply $\mathbb{F}$, for the set of all fixed-precision numbers with p total bits and s fractional bits.

We represent negative numbers using two's complement. If $b \in [p]$, the b -th bit of a fixed-precision number x , written $\langle x \rangle_b$, is defined as

$$\langle x \rangle_b = \begin{cases} 1 & \text{if } \lfloor x/2^{b-s-1} \rfloor \text{ is odd} \\ 0 & \text{otherwise.} \end{cases}$$

In two's complement we note that

$$x = -2^p \cdot \langle x \rangle_p + \sum_{b \in [p-1]} 2^{b-s-1} \cdot \langle x \rangle_b.$$

If x is a real number, we write $\text{round}_{\mathbb{F}}(x)$ or simply $\text{round}(x)$ for the greatest element of $\mathbb{F}$ less than or equal to x .

Definition B.2. A (future-masked) $(1, \log n)$ -precision transformer of depth k is a function $T: \Sigma^* \rightarrow \mathbb{F}$, defined in terms of functions

$E: \Sigma \rightarrow \mathbb{F}^d$	word embedding
$W_Q^{(\ell)}, W_K^{(\ell)}, W_V^{(\ell)}: \mathbb{F}^d \rightarrow \mathbb{F}^d \quad \ell = 1, \dots, k$	self-attention
$f^{(\ell)}: \mathbb{F}^d \rightarrow \mathbb{F}^d \quad \ell = 1, \dots, k$	feed-forward
$W_{\text{out}}: \mathbb{F}^d \rightarrow \mathbb{F}$	output unembedding.

On input w , $T(w)$ is computed as follows:

$$\mathbf{h}_i^{(0)}(w) = E(w_i) \quad (4)$$

For $\ell = 1, \dots, k$:

$$\mathbf{q}_i^{(\ell)}(w) = W_Q^{(\ell)}(\mathbf{h}_i^{(\ell-1)}(w)) \quad (5)$$

$$\mathbf{k}_i^{(\ell)}(w) = W_K^{(\ell)}(\mathbf{h}_i^{(\ell-1)}(w)) \quad (6)$$

$$\mathbf{v}_i^{(\ell)}(w) = W_V^{(\ell)}(\mathbf{h}_i^{(\ell-1)}(w)) \quad (7)$$

$$\mathbf{s}_{ij}^{(\ell)}(w) = \mathbf{q}_i^{(\ell)}(w) \cdot \mathbf{k}_j^{(\ell)}(w) \quad (8)$$

$$A_i^{(\ell)}(w) = \sum_{j=1}^i \text{round}\left(\exp\left(\mathbf{s}_{ij}^{(\ell)}(w)\right)\right) \mathbf{v}_j^{(\ell)}(w) \quad (9)$$

$$B_i^{(\ell)}(w) = \sum_{j=1}^i \text{round}\left(\exp\left(\mathbf{s}_{ij}^{(\ell)}(w)\right)\right) \quad (10)$$

$$\mathbf{c}_i^{(\ell)}(w) = \text{round}\left(\frac{A_i^{(\ell)}(w)}{B_i^{(\ell)}(w)}\right) \quad (11)$$

where Equation (11) evaluates to the average of all $\mathbf{v}_j^{(\ell)}$ if the denominator is 0, and finally

$$\mathbf{h}_i^{(\ell)}(w) = f^{(\ell)}\left(\mathbf{c}_i^{(\ell)}(w) + \mathbf{h}_i^{(\ell-1)}(w)\right) \quad (12)$$

$$T(w) = W_{\text{out}}\left(\mathbf{h}_{|w|}^{(k)}(w)\right). \quad (13)$$

We say that T accepts w if $T(w) > 0$.

To measure the size of a transformer, we assume that the constants in all of its defining functions are given in binary and the precision is given in unary.

Note crucially that Equation (11) is written so that even if $i \gg 2^s$, it is still possible to obtain nonzero values.

Definition B.3. A (future-masked) $(1, 1)$ -precision transformer is defined similarly to a $(1, \log n)$ -precision transformer except the attention weights are rounded before using them to take a weighted average of the value vectors:

$$\begin{aligned} A_{i,j}^{(\ell)}(w) &= \text{round}\left(\exp\left(\mathbf{s}_{ij}^{(\ell)}(w)\right)\right) \\ \alpha_{i,j}^{(\ell)}(w) &= \text{round}\left(\frac{A_{i,j}^{(\ell)}(w)}{B_i^{(\ell)}(w)}\right) \\ \mathbf{c}_i^{(\ell)}(w) &= \text{round}\left(\sum_{j=1}^i \alpha_{i,j}^{(\ell)}(w) \mathbf{v}_j^{(\ell)}(w)\right) \end{aligned}$$

We will often make use of the following operator in C-RASP, which does not increase its expressive power or affect the depth of formulas, but saves space when writing.

$$(\phi ? t_{\text{then}} : t_{\text{else}})^{w,i} = \begin{cases} t_{\text{then}} & w, i \models \phi \\ t_{\text{else}} & w, i \not\models \phi \end{cases}$$

Lemma B.4 (Yang & Chiang 2024). Any formula ϕ of C-RASP that uses the $?$ operator can be converted into a formula that does not use the $?$ operator, defines the same language as ϕ , and has the same depth as ϕ .

Proof. Any comparison formula involving the $?$ operator can be written in the form

$$(\psi_{\text{if}} ? t_{\text{then}} : t_{\text{else}}) + \sum_{\ell \in [m]} t_{\ell} \geq C,$$

which can be rewritten as

$$\left(\psi_{\text{if}} \wedge t_{\text{then}} + \sum_{\ell \in [m]} t_{\ell} \geq C \right) \vee \left(\neg \psi_{\text{if}} \wedge t_{\text{else}} + \sum_{\ell \in [m]} t_{\ell} \geq C \right).$$

This rule can be used iteratively to rewrite all the $?$ operators out of a formula. $\square$

B.2. Previous Work

Previous work by Li & Cotterell (2025) characterized the expressivity of $(1, 1)$ -precision transformers in terms of temporal logic, and furthermore provided empirical evidence that transformers length-generalized very well on this class of languages.

Theorem B.5 (Li & Cotterell, 2025, Thm. 3.2). *Every $(1, 1)$ -precision transformer can be simulated by a $\text{TL}[\Diamond]$ formula.*

Theorem B.6 (Li & Cotterell, 2025, Thm. 3.3). *Every $\text{TL}[\Diamond]$ formula can be simulated by a $(1, 1)$ -precision transformer.*

Their construction did not explicitly compute the size bounds, but a careful reading shows that a singly exponential blowup occurs when simulating $(1, 1)$ -precision transformers in $\text{TL}[\Diamond]$, and a polynomial blowup occurs the other way around.

Our contribution is to derive precise size bounds in converting between C-RASP₊ and $(1, 1)$ -precision transformers, which also turn out to be singly exponential and polynomial, in the same way. However, an additional point we note is that the depth of the $\text{TL}[\Diamond]$ formula turns out to be exponential in the size of the transformer, while the depth of the C-RASP₊ formula is linear in the depth of the transformer. This perspective may be used to shed more precise insight on the sizes of transformers, similar to how Yang et al. (2025) used a depth-preserving equivalence between C-RASP and $(1, \log n)$ -precision transformers in order to derive a strict depth hierarchy for transformers.

B.3. Fixed-Precision Transformers to Logic

Definition B.7. Let $\mathbf{x}_i : \Sigma^* \rightarrow \mathbb{F}^d$ be a sequence of activation vectors depending on the input string w . We will often omit the dependence on w and just write $\mathbf{x}_i$ instead of $\mathbf{x}_i(w)$ whenever it is clear from context.

We say that a set of C-RASP₊ predicates $P_{\langle \mathbf{x}_c \rangle_b}$ (for $c \in [d]$ and $b \in [p]$) simulates $\mathbf{x}$ if for every $c \in [d]$ and $b \in [p]$,

$$w, i \models P_{\langle \mathbf{x}_c \rangle_b} \iff \langle \mathbf{x}_i(w) \rangle_c = 1.$$

We first define what it means for a $(1, \log n)$ -precision transformer and a formula to simulate each other.

Definition B.8. We say that a C-RASP formula ϕ *simulates* a $(1, \log n)$ -precision transformer T if, for all $w \in \Sigma^*$,

$$w, i \models \phi \iff T(\langle \text{BOS} \rangle \cdot w)_i > 0.$$

In other words, $w \models \phi$ if and only if T accepts $\langle \text{BOS} \rangle \cdot w$.

We say that a $(1, \log n)$ -precision transformer T with depth k and dimension d *simulates* a formula ϕ of C-RASP T if, for all $w \in \Sigma^*$,

$$T(\langle \text{BOS} \rangle \cdot w)_i > 0 \iff w, i \models \phi.$$

Again, T accepts $\langle \text{BOS} \rangle \cdot w$ if and only if $w \models \phi$.

We first state a utility lemma which we use repeatedly to complete the main theorem. It says that any positionwise function on fixed-precision numbers can be defined in propositional logic.

Lemma B.9. *Let $f : \mathbb{F}^d \rightarrow \mathbb{F}^d$ be a function. Let $\mathbf{x}_i : \Sigma^* \rightarrow \mathbb{F}^d$ be a sequence of activation vectors simulated by predicates $P_{\langle \mathbf{x}_c \rangle_b}$ for $c \in [d]$ and $b \in [p]$. Then there exist $\text{TL}[\Diamond]$ programs*

- (a) $P_{\langle f(\mathbf{x}_c) \rangle_b}$ *simulating* $f(\mathbf{x}_i)$, with size $O(p^2 d^2 2^{pd})$.

(b) $P_{\mathbf{x}=\mathbf{y}}$, where $\mathbf{y} \in \mathbb{F}^d$ (not depending on w) such that

$$w, i \models P_{\mathbf{x}=\mathbf{y}} \iff \mathbf{x}_i(w) = \mathbf{y}$$

with size pd .

Proof. (a) For each $c \in [d]$ and $b \in [p]$ we define the program line:

$$P_{\langle f(\mathbf{x})_c \rangle_b} := \bigvee_{\substack{\mathbf{v} \in \mathbb{F}^d \\ \langle f(\mathbf{v})_c \rangle_b = 1}} \bigwedge_{\substack{c' \in [d], b' \in [p] \\ \langle \mathbf{v}_{c'} \rangle_{b'} = 1}} P_{\langle \mathbf{x}_{c'} \rangle_{b'}}.$$

The size of each such line is at most $pd 2^{pd}$. As there are pd such lines, this increases the size of the program (additively) by $O(p^2 d^2 2^{pd})$.

(b) Define

$$P_{\mathbf{x}=\mathbf{y}} := \bigwedge_{c \in [d], b \in [p]} (P_{\langle \mathbf{x}_c \rangle_b} \leftrightarrow \langle \mathbf{y}_c \rangle_b).$$

The size of this line is pd .

Overall, the size of the program is $O(p^2 d^2 2^{pd})$. Note that this is in $O(2^{\text{poly}(x,y)})$. We note this can be generalized to multi-ary functions with still an exponential bound. $\square$

Theorem B.10. For a $(1, 1)$ -precision transformer T with bit precision p , hidden dimension d , and depth L , there exists a C-RASP₊ program P with:

- Size $O(|\Sigma| L p^2 d 2^{p^3 d})$
- Depth $O(L)$
- Precision $O(p)$
- Girth $O(p)$

Proof. We will inductively construct a C-RASP₊ program that simulates T , while maintaining an upper bound on the size of the program. We will show for every activation $\mathbf{h}_{i,c}^\ell$ there exists a program line $P_{\langle \mathbf{h}_c^\ell \rangle_b}$ that simulates $\mathbf{h}_{i,c}^\ell$. To make our presentation more concise, we will often write many C-RASP₊ definitions condensed as a large formula in a single line. To rewrite it in proper C-RASP₊ semantics, we can expand it into a program with size upper bounded by the number of symbols occurring in the formula.

The construction proceeds by induction on ℓ . For $\ell = 0$, the transformer consists of just a word embedding layer, which we simulate using the following program:

$$P_{\langle \mathbf{h}_c^{(0)} \rangle_b} := \bigvee_{\substack{\sigma \in \Sigma \\ \langle E(\sigma)_c \rangle_b = 1}} \sigma.$$

Here there are pd program lines, each of size $O(|\Sigma|)$. This program has precision $O(1)$, size $P(pd|\Sigma|)$, and depth $O(1)$.

Now suppose the inductive hypothesis holds for layer L , and consider layer $(L + 1)$. That is, first assume we have defined program lines $P_{\langle \mathbf{h}_c^{(L-1)} \rangle_b}$ for each $b \in [p]$ and $c \in [d]$ simulating the activations coming out of the $(\ell - 1)$ -th layer, with:

- Size $O(|\Sigma|(L - 1)p^2 d 2^{p^3 d})$
- Depth $O(L - 1)$
- Precision $O(p)$

- Girth $O(p)$

To simulate the next layer, we want program lines $P_{\langle \mathbf{q}_c^{(L)} \rangle_b}$, $P_{\langle \mathbf{k}_c^{(L)} \rangle_b}$, and $P_{\langle \mathbf{v}_c^{(L)} \rangle_b}$ such that:

$$\begin{aligned} w, i &\models P_{\langle \mathbf{q}_c^{(L)} \rangle_b} \iff \langle \mathbf{q}_{i,c}^{(L)}(w) \rangle_b = 1 \\ w, i &\models P_{\langle \mathbf{k}_c^{(L)} \rangle_b} \iff \langle \mathbf{k}_{i,c}^{(L)}(w) \rangle_b = 1 \\ w, i &\models P_{\langle \mathbf{v}_c^{(L)} \rangle_b} \iff \langle \mathbf{v}_{i,c}^{(L)}(w) \rangle_b = 1. \end{aligned}$$

This allows us to simulate the attention weight matrices. These program lines can be defined using [Lemma B.9](#), adding $O(p^2 d^2 2^{p^2 d})$ to the program size but not changing the depth, girth, or precision.

Next, we want to compute the summands A and B in the numerator and denominator of attention. One potential difficulty is that summands retrieve values from two positions (i and j) simultaneously, whereas a C-RASP₊ program can retrieve a value from one position at a time. However, since $\mathbf{q}_i$ can only take on finitely many values, we can simulate retrieval by enumerating all these possible values.

To perform this enumeration we write program lines $P_{\mathbf{q}^{(L)}=\mathbf{q}}$, $P_{\mathbf{k}^{(L)}=\mathbf{k}}$, and $P_{\mathbf{v}^{(L)}=\mathbf{v}}$ for each $\mathbf{q}, \mathbf{k}, \mathbf{v} \in \mathbb{F}^d$ (not depending on w) such that:

$$\begin{aligned} w, i &\models P_{\mathbf{q}^{(L)}=\mathbf{q}} \iff \mathbf{q}_i^{(L)}(w) = \mathbf{q} \\ w, j &\models P_{\mathbf{k}^{(L)}=\mathbf{k}} \iff \mathbf{k}_j^{(L)}(w) = \mathbf{k} \\ w, j &\models P_{\mathbf{v}^{(L)}=\mathbf{v}} \iff \mathbf{v}_j^{(L)}(w) = \mathbf{v} \end{aligned}$$

This is also achieved using [Lemma B.9](#), adding $O(p^2 d^2 2^{p^2 d})$ to the program size but not changing the depth, girth, or precision. Then, we compute $\mathbf{s}_{ij}^{(L)}(w)$, found in the numerator of the attention scores. This depends on the value of $\mathbf{q}_i$, so for each $\mathbf{q} \in \mathbb{F}^d$ we will define a program line $A_{\mathbf{q},b}^{(L)}$ such that:

$$w, j \models A_{\mathbf{q},b}^{(L)} \iff \left\langle \text{round}\left(\exp\left(\mathbf{q} \cdot \mathbf{k}_j^{(L)}(w)\right)\right) \right\rangle_b = 1.$$

We write these formulas as

$$A_{\mathbf{q},b}^{(L)} := \bigvee_{\substack{\mathbf{k} \in \mathbb{F}^d : \\ \langle \text{round}(\exp(\mathbf{q} \cdot \mathbf{k})) \rangle_b = 1}} P_{\mathbf{k}^{(L)}=\mathbf{k}}$$

These add $O(2^{pd})$ to the program size but don't change the depth, girth, or precision.

Then we use this to compute the summand $B_i^{(L)}(w) = \sum_{j=1}^i \text{round}\left(\exp\left(\mathbf{s}_{ij}^{(L)}(w)\right)\right)$. The idea is to fix $\mathbf{q}_i^{(L)}$ for each $\mathbf{q} \in \mathbb{F}^d$ and compute what $B_i^{(L)}(w)$ would be as a counting term, and then perform a disjunction over all $\mathbf{q}$ to truncate these values back to fixed precision. First, for each $\mathbf{q} \in \mathbb{F}^d$ we can fix $\mathbf{q}_i^{(L)}$ and compute the value as a term $C_{B_i^{(L)}} \in \mathbb{N}$ such that:

$$C_{\mathbf{q},B_i^{(L)}}^{w,i} = 2^s \sum_{j \leq i} \text{round}\left(\exp\left(\mathbf{q} \cdot \mathbf{k}_j^{(L)}(w)\right)\right).$$

Because each term is always nonnegative, we can ignore the most significant bit and write the counting term as follows:

$$C_{\mathbf{q},B_i^{(L)}} := \sum_{b \in [p-1]} 2^{b+s-1} \cdot \# \left[A_{\mathbf{q},b}^{(L)} \right]$$

Then, we perform a disjunction over all $\mathbf{q}$ to compute $B_i^{(L)}(w)$ and then round to fixed-precision. Note that we truncate any values above the maximum fixed-precision number, as per the rounding rules defined. The program line is as follows:

$$B_b^{(L)} := \bigvee_{\mathbf{q} \in \mathbb{F}^d} \left(P_{\mathbf{q}, B^{(L)} = \mathbf{q}} \left(\bigvee_{\substack{x \in \mathbb{F} \\ x < 2^{p-s-1} \\ \langle x \rangle_b = 1}} (C_{\mathbf{q}, B^{(L)}} = 2^s \cdot x) \vee (C_{\mathbf{q}, B^{(L)}} \geq 2^s \cdot x) \right) \right).$$

Each $C_{\mathbf{q}, B^{(L)}}$ has size $O(p)$, precision $O(p)$, girth $O(p)$, and depth $O(1)$. Thus in total all the $B_b^{(L)}$ program lines contribute size $O(p^2 2^{p^2 d})$, precision $O(p)$, girth $O(p)$, and depth $O(1)$.

The next step is to simulate α_{ij} . To begin, using [Lemma B.9](#) we can write program lines $P_{B^{(L)}=v}$ for $v \in \mathbb{F}$ such that

$$w, i \models P_{B^{(L)}=v} \iff B^{(L)} = v$$

This adds $O(p^2 d^2 2^{p^2 d})$ to the program size without changing the depth, girth, or precision. To remove the dependence of α_{ij} on multiple positions, we again need to enumerate over all $\mathbf{q} \in \mathbb{F}^d$. We use the notation $\alpha_{\mathbf{q}, j} = \text{round} \left(\frac{\text{round}(\exp(\mathbf{q} \cdot \mathbf{k}_j^{(L)}(w)))}{\sum_{j=1}^i \text{round}(\exp(\mathbf{q} \cdot \mathbf{k}_j^{(L)}(w)))} \right)$. Then for each $\mathbf{q}$ we can write formulas $P_{\langle \alpha_{\mathbf{q}} \rangle_b}$ such that:

$$w, j \models P_{\langle \alpha_{\mathbf{q}} \rangle_b} \iff \langle \alpha_{\mathbf{q}, j}(w) \rangle_b = 1.$$

This uses the multi-ary generalization of [Lemma B.9](#), where $\alpha_{\mathbf{q}, j} = f(A_{\mathbf{q}, b}^{(L)}, B_b^{(L)})$. This adds $O(p^2 d^2 2^{p^2 d})$ to the program size without changing the depth, girth, or precision.

Now that the attention scores have been simulated, the next step is to simulate the weighted average of the values by the attention scores. As observed by [Li & Cotterell \(2025\)](#) and [Merrill & Sabharwal \(2023\)](#), there are finitely many positions where this can be nonzero. We label this bound $M^{(L)}$. For this we want to write a count $C_{\mathbf{q}, M^{(L)}}$ such that

$$(C_{\mathbf{q}, M^{(L)}})^{w, i} = |\{j \in [|w|] : \alpha_{\mathbf{q}, j} > 0\}|$$

First, we write a formula $P_{\alpha_{\mathbf{q}} > 0}$ for each $\mathbf{q} \in \mathbb{F}^d$ such that:

$$w, j \models P_{\alpha_{\mathbf{q}} > 0} \iff \alpha_{\mathbf{q}, j}(w) > 0$$

This can be achieved using [Lemma B.9](#), $O(p^2 d^2 2^{p^2 d})$ to the program size without changing the depth, girth, or precision. Then we write a count term

$$C_{\mathbf{q}, M^{(L)}} := \overline{\#} [P_{\alpha_{\mathbf{q}} > 0}]$$

The exact value of $C_{\mathbf{q}, M^{(L)}}$ must range from 0 to 2^s ([Li & Cotterell, 2025](#), Proposition B.6). So there exist formulas $P_{M^{(L)}=M}$ for $M \in [0, 2^s]$ such that

$$w, i \models P_{M^{(L)}=M} \iff M^{(L)} = M.$$

We can then write this as follows:

$$P_{M^{(L)}=M} := \bigvee_{\mathbf{q} \in \mathbb{F}^d} C_{\mathbf{q}, M^{(L)}} = M.$$

Each $C_{\mathbf{q}, M^{(L)}}$ has size $O(1)$, girth $O(1)$, depth (1) , and precision $O(1)$. When plugged into each $P_{M^{(L)}=M}$, this results in an overall contribution of size $O(2^{pd})$, precision $O(1)$, depth $O(1)$, and girth $O(1)$.

Next, we need to check if $\text{round}\left(\sum_{j=1}^i \alpha_{\mathbf{q},j} \mathbf{v}_j^{(L)}(w)\right) = \mathbf{x}$ for $\mathbf{x} \in \mathbb{F}^d$. The reason this is not straightforward is that we may have to compute a summation of negative-valued $\mathbf{v}$ (which is not definable in C-RASP_+).

The solution we use is to turn the summation nonnegative by adding 2^p to any nonzero summand. Since there are exactly $M^{(L)}$ positions receiving nonzero attention, we add exactly $M^{(L)} \cdot 2^p$ to the total. Stated more formally, we simulate the output of attention using only the summation of nonnegative values:

$$\text{round}\left(\sum_{1 \leq j \leq i} \alpha_{\mathbf{q},j} \left(\mathbf{v}_j^{(L)}(w)_c\right)\right) = \mathbf{x}_c \iff \text{round}\left(\sum_{\substack{1 \leq j \leq i: \\ \alpha_{\mathbf{q},j} > 0}} \alpha_{\mathbf{q},j} \left(\mathbf{v}_j^{(L)}(w)_c\right) + 2^p\right) = \mathbf{x}_c + M_{\mathbf{q}}^{(L)} \cdot 2^p.$$

To accomplish this, first we write program lines $P_{\langle \alpha_{\mathbf{q}} \mathbf{v}_c^{(L)} \rangle_b}$ to define the bits of $\langle \alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)} \rangle_b$. That is:

$$w, j \models P_{\langle \alpha_{\mathbf{q}} \mathbf{v}_c^{(L)} \rangle_b} \iff \langle \alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)} \rangle_b = 1$$

This is achieved again using [Lemma B.9](#), again adding $O(p^2 d^2 2^{p^2 d})$ to the program size without changing the depth, girth, or precision.

Then, for the summation, we can sum the first $p-1$ bits as usual (because they are all positive). Because we add a 2^p whenever $\alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)}$, this interacts with the summation of -2^p in p -th bit. Specifically, to compute the sum:

- If $\alpha_{\mathbf{q}} = 0$, we add 0
- If $\alpha_{\mathbf{q}} > 0$ and $\langle \alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)} \rangle_p = 1$, we add 0
- If $\alpha_{\mathbf{q}} > 0$ and $\langle \alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)} \rangle_p = 0$, we add 2^p

Thus we just need to check $P_{\alpha_{\mathbf{q}} > 0} \wedge \neg P_{\langle \alpha_{\mathbf{q}} \mathbf{v}_c^{(L)} \rangle_b}$ to determine how to perform the summation in the p -th bit.

Thus we can compute the modified sum as follows:

$$C_{\mathbf{q}, \mathbf{c}_c^{(L)}} := 2^p \cdot \overset{\sim}{\#} \left[P_{\alpha_{\mathbf{q}} > 0} \wedge \neg P_{\langle \alpha_{\mathbf{q}} \mathbf{v}_c^{(L)} \rangle_b} \right] + \sum_{b \in [p-1]} 2^{b-s-1} \cdot \overset{\sim}{\#} \left[P_{\langle \alpha_{\mathbf{q}} \mathbf{v}_c^{(L)} \rangle_b} \right]$$

Finally we round to fixed precision. For $\mathbf{x} \in \mathbb{F}$ we write a program line checking if $\mathbf{c}_c^{(L)} = \mathbf{x}$ as follows. Crucially, we know there are exactly M positions where $\alpha_{\mathbf{q},j} \mathbf{v}_{j,c}^{(L)}$ is not zero in every bit. So we can check the value of $P_{M_{\mathbf{q}}=M}$ and use that to add the correction factor to the sum.

$$P_{\mathbf{c}_c^{(L)} = \mathbf{x}} := \bigvee_{\mathbf{q} \in \mathbb{F}^d} P_{\mathbf{q}^{(L)} = \mathbf{q}} \left(\bigvee_{M \in \mathbb{F}_{s,0}} \left(P_{M_{\mathbf{q}}=M} \wedge \left(\bigvee_{\substack{x \in \mathbb{F} \\ x < 2^{p-s-1} \\ \langle x \rangle_b = 1}} (C_{\mathbf{c}_c^{(L)}} = 2^s \cdot x + M \cdot 2^p) \vee (C_{\mathbf{c}_c^{(L)}} \geq 2^s \cdot x + M \cdot 2^p) \right) \right) \right)$$

Each $C_{\mathbf{q}, \mathbf{c}_c^{(L)}}$ has size $O(p)$, girth $O(p)$, depth $O(1)$, and precision $O(p)$. When plugged into each $P_{\mathbf{c}_c^{(L)} = \mathbf{x}}$, there is a total contribution of size $O(pd2^{p^3 d})$, depth $O(1)$, girth $O(p)$, and precision $O(p)$.

Finally, to simulate $\mathbf{h}_i^{(L)}(w) = f^{(L)}\left(\mathbf{c}_i^{(L)}(w) + \mathbf{h}_i^{(L-1)}(w)\right)$ we use the multi-ary case of [Lemma B.9](#) once more, as we have simulated $\mathbf{c}_i^{(L)}$ and $\mathbf{h}_i^{(L-1)}$. This again adds $O(p^2 d^2 2^{p^2 d})$ to the program size without changing the depth, girth, or precision.

Overall we have increased the size additively by $O(p^2 d 2^{p^3 d})$, increased the depth additively by $O(1)$, left the precision at $O(p)$, and left the girth at $O(p)$. This results in a program with

- Size $O(|\Sigma| L p^2 d 2^{p^3 d})$
- Depth $O(L)$
- Precision $O(p)$
- Girth $O(p)$

□

B.4. Logic to Fixed-Precision Transformers

Theorem B.11. *Let ϕ be a C-RASP₊ program with precision p , depth L , and size d . There exists a $(1, 1)$ -precision transformer T with precision $O(p)$, depth $O(L)$, and hidden dimension $O(d)$ which simulates ϕ .*

Proof. All cases follow similarly as described in Yang et al. (2025); Li & Cotterell (2025) except for $\phi = \sum_{t \in \mathcal{T}} \alpha_t t \sim k$, where $\alpha_i, k \in \mathbb{N}$ and $\sim \in \{<, \leq, =, \geq, >\}$. We will just discuss the case of $\sim \in \{\geq\}$, from which all others are definable using boolean functions and adding a constant +1.

Consider a subformula $\phi = \sum_{t \in \mathcal{T}} \alpha_t t \geq k$. We can separate out the constants terms to rewrite it in the form $\phi = C + \sum_{f \in \mathcal{F}} \alpha_f \#[\phi_f] \geq k$, where $C \in \mathbb{N}$. Assume that previous layers have computed $\mathbb{I}[w, j \models \phi_t]$ for $t \in \mathcal{T}$ at all positions $j \in [n]$. We want to construct a new layer that computes $\mathbb{I}[w, i \models \phi]$ at all positions $i \in [n]$.

We pick a fixed-precision representation $\mathbb{F}$ which contains $\frac{1}{k}$ as well as k , and also contains a value x such that $\text{round}(\exp(x)) = 0$. This is always possible.

Using constructions from Yang et al. (2026b), it is possible to set W_Q, W_K , and W_V so that:

$$\begin{aligned} \text{round}(\exp(s_{ij})) &= \begin{cases} 1 & w, j \models Q_{\langle \text{BOS} \rangle} \\ 1 & w, j \models \bigvee_{t \in \mathcal{T}} \phi_t \\ 0 & \text{otherwise} \end{cases} \\ \mathbf{v}_j &= \begin{cases} -(k-1) + C & w, j \models Q_{\langle \text{BOS} \rangle} \\ \sum_{t \in \mathcal{T}} \mathbb{I}[w, j \models \phi_t] & \text{otherwise} \end{cases} \end{aligned}$$

Then, after plugging into the definition of attention in Equation (11) we get that:

$$\mathbf{c}_i^{(L)}(w) = \text{round} \left(\frac{-(k-1) + \sum_{j=1}^i \sum_{t \in \mathcal{T}} \mathbb{I}[w, j \models \phi_t]}{|\{j \in [|w|] : \mathbb{I}[w, j \models \bigvee_{t \in \mathcal{T}} \phi_t]\}| + 1} \right)$$

We observe that $\mathbf{c}_i^{(L)} \geq 0 \iff w, i \models \sum_{t \in \mathcal{T}} \alpha_t t \geq k$. Thus, $\mathbf{c}_i^{(L)}$ rounds to -2^{-s} or below if ϕ is false, and rounds to 0 or above if ϕ is true. We can then use the FFNN f to map these two cases to 0 or 1, respectively. This adds one layer, some constant number of hidden dimensions, and uses the same precision p .

□

C. Depth 2 Is Sufficient

Proposition C.1. *There is a satisfiability-preserving reduction from C-RASP to C-RASP with depth at most 2.*

The proof of this proposition is similar to the so-called *Scott's Normal Form* for first-order logic with two variables (Grädel et al., 1997). Given $\varphi \in \text{C-RASP}$, we construct φ' with $\text{dp}(\varphi') = 1$ and a set $S := S_\varphi$ of “basic” formulas such that φ is satisfiable iff $\varphi' \wedge \bigwedge_{\psi \in S} \psi$ is satisfiable. The proof goes by induction on the depth of φ . If $\text{dp}(\varphi) \leq 1$, then $S = \{\varphi\}$. If $\text{dp}(\varphi) > 2$, then for each $\#\psi$ with $\text{dp}(\psi) = 1$ occurring in φ , we have by induction constructed ψ' with $\text{dp}(\psi') = 1$ and the set S_ψ of basic formulas. Starting with $S_\varphi = \emptyset$, we introduce a new proposition P_ψ and do the following: (i) replace ψ in φ by P_ψ , and (ii) add to S_φ the formula $\#((P_\psi \wedge \neg\psi') \vee (\neg P_\psi \wedge \psi')) = 0$. This results in a formula φ' with $\text{dp}(\varphi') = 1$ and S_φ such that φ is satisfiable iff $\varphi' \wedge \bigwedge_{\psi \in S_\varphi} \psi$ is satisfiable.